

Day 12: Local decision-making and MQTT

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 3, 2026

- ▶ On Day 11 the boards sent video. A system that sends only results needs less bandwidth and keeps the images on the device. Today the boards send small messages: a class, a reading, a command.
- ▶ Some decisions cannot wait for a server: a door that must close, a motor that must stop. The device must decide alone, also when the network or the internet fails.
- ▶ MQTT is the common protocol for such messages. A broker receives each message and gives it to the devices that asked for its topic. The protocol has settings for lost messages, lost devices, and the last value.
- ▶ In the lab you install a broker on the Raspberry Pi, send IMU readings and model results from the XIAO, apply a local rule that sends a command back, and show that the system works with no internet and loses no message.

After this day you can:

- ▶ Design a cascade that makes decisions locally.
- ▶ Explain MQTT topics, quality of service levels, retained messages, and the last will message.
- ▶ Publish inference results from a microcontroller.
- ▶ Build a system that continues to work without an internet connection.

Part 1 Local decision-making

Exercise: write the state machine for a door monitor with two sensors

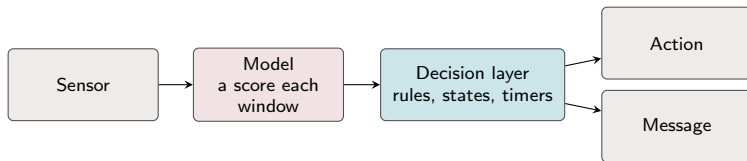
Part 2 MQTT

Exercise: design the topic tree for ten devices in two rooms

Part 3 Edge-to-cloud design

Exercise: calculate the data of video and of detections for one day

A model gives a score, a program decides



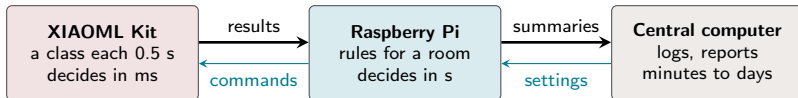
- ▶ A model gives a score for each window: “person 0.62”. It does not say what the system must do.
- ▶ The decision layer turns the scores into actions: switch on a light, stop a motor, send an alert.
- ▶ It is ordinary code: rules, a state machine, and timers. You can read it, test it, and change it with no new training.
- ▶ The scores change from window to window. The decision layer must stay stable when a score jumps for one window.

Key point: A good model with a bad decision layer gives a bad system.

Pattern	Trade-off	Choose when	In this course
Train-serve split	Training cost against inference latency	Training needs scale that inference does not need	Train on a laptop, run on the XIAO (Days 3 to 5)
Hierarchical processing	Local autonomy against global optimization	The data is more than the link carries; decisions on several time scales	XIAO, then Raspberry Pi, then a central computer (this lab)
Progressive deployment	Model quality against deployment reach	The same model at several capability levels	One detector in float32 and int8 (Day 8)

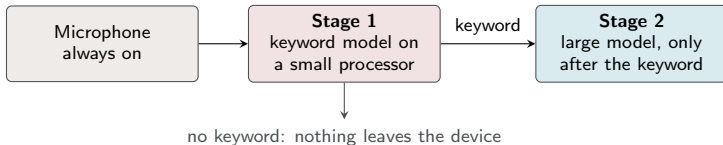
- ▶ A hybrid system puts each stage on the tier where it meets its limits best: the time, the energy, the memory, or the data.
- ▶ The tiers must agree on the model version, the features, and the pre-processing. Else the same input gives two answers.

Source: *Machine Learning Systems*, Volume I, chapter “ML Systems”, section “Hybrid Architectures”.



- ▶ Each tier decides what it can decide alone, and sends less data upward: a class, not the audio; a count, not the classes.
- ▶ A higher tier changes the rules of a lower tier: a threshold, a schedule, a model version. It does not take each decision.
- ▶ When the link to a higher tier fails, the lower tiers continue. This is the lab of today.

A cascade: a small model wakes a large one



- ▶ A voice assistant listens all the time with a keyword model on a small processor. Only after the keyword does it send the audio to a large model.
- ▶ The small stage runs all the time, so it must use little energy. The large stage runs seldom, so it can be slow and expensive.
- ▶ The first stage also protects privacy: most audio never leaves the device.
- ▶ Errors of stage 1 set the behaviour: a missed keyword ignores the user; a false keyword sends audio for nothing.

Source: Kit lab "Keyword Spotting" of *Machine Learning Systems*.

Which decisions must stay local

Reason	Question	Example
Deadline	Is the deadline shorter than the round trip to the server, with resends?	Stop a motor within 50 ms
No network	Must the decision work when the link fails?	Open a door for a person
Safety	Can a late or a missing decision hurt somebody?	Switch off a heater that is too hot
Privacy	Does the decision need data that must not leave?	Count the people in a room from the video
Volume	Is the input much larger than the result?	Classify 16 000 audio samples each second

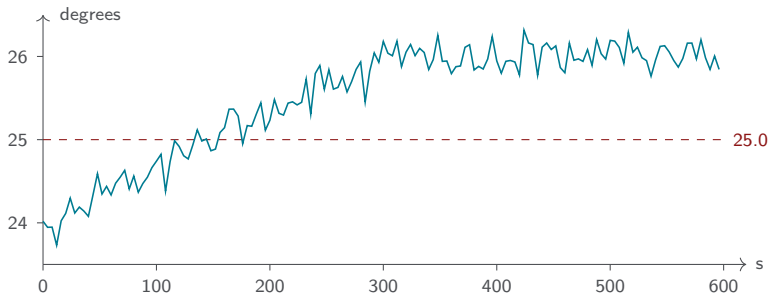
- ▶ One “yes” is enough: the decision stays on the device.
- ▶ On Day 11, a server on the internet answered in 69 ms. A lost TCP segment waits at least 200 ms on Linux. A deadline of 50 ms cannot use the internet.
- ▶ The cloud keeps the decisions that need much data and no fast answer: reports, retraining, new thresholds.

Source: Day 11, Part 1.

```
def decide(scores):                                # scores: {"person": 0.62, ...}
    label = max(scores, key=scores.get)
    if scores[label] < 0.5:                         # not sure: no action
        return "none"
    if label == "fire":                            # a dangerous class wins
        return "alarm"
    if label == "person":
        return "light_on"
    return "none"
```

- ▶ **A threshold:** act only when the score is high enough. The threshold sets the balance of missed events and false events.
- ▶ **A priority:** a dangerous class wins over a normal class, also with a lower score.
- ▶ **A default:** “no action” when the model is not sure, or when the class is unknown.
- ▶ A rule on one window reacts to each noisy window. The next frames show the effect and two fixes: hysteresis and filters in time.

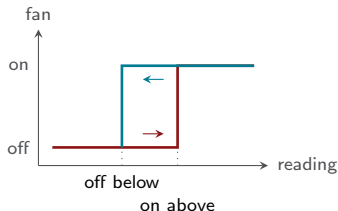
A noisy reading near one threshold



- ▶ A temperature rises from 24 to 26 degrees in 300 s. One reading each second, with noise of 0.15 degrees. A fan runs above 25.0 degrees.
- ▶ The true value crosses 25.0 degrees at 150 s. The noisy reading crosses the line many times near this point.
- ▶ With one threshold, the fan switches 25 times in 600 s: each switch wears the relay and the motor.

Source: Simulation of this course (synthetic signal, seed 0). One point each 4 s is shown.

Hysteresis: two thresholds

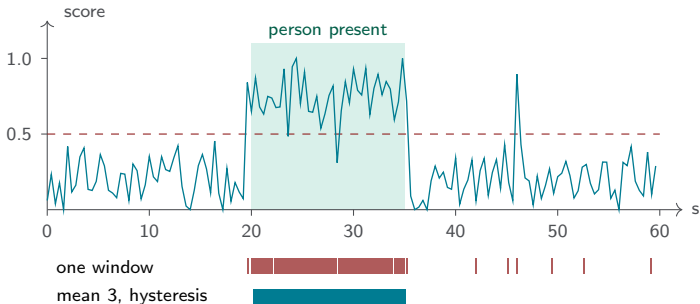


Rule	Switches	First on
Threshold 25.0	25	133 s
Band ± 0.2	1	164 s
Band ± 0.4	1	187 s

The same 600 readings. The true value crosses 25.0 at 150 s.

- ▶ Switch on above the upper threshold, and off only below the lower one. Noise inside the band changes nothing.
- ▶ Select the band wider than the noise: here about 2 to 3 times the standard deviation of 0.15 degrees.
- ▶ The cost is delay: the band of ± 0.4 switched on 37 s after the true crossing. One threshold switched on 17 s before it, because of the noise.

Source: Simulation of this course (synthetic signal, seed 0).



- ▶ A “person” score, 10 windows each second. A person is present from 20 s to 35 s. With no person, 6 single windows have a false high score.
- ▶ The bars show when a rule says “person”. The rule on one window says it 15 times in this run, 8 times with no person.

Source: Simulation of this course (synthetic scores, seed 0). The line shows one window each 0.4 s; the bars use all windows.

Simulated: five rules on the same scores

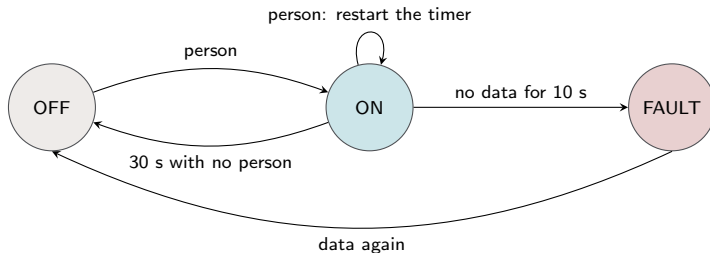
Rule	Switch-on	False	Delay on	Delay off
One window above 0.5	16.11	8.60	0.04 s	0.10 s
Mean of 3 above 0.5	2.32	1.09	0.11 s	0.09 s
3 of 5 above 0.5	1.07	0.00	0.21 s	0.20 s
Hysteresis, one window	8.50	6.11	0.17 s	0.01 s
Hysteresis, mean of 3	1.06	0.06	0.17 s	0.14 s

Mean of 100 runs (seeds 0 to 99). Hysteresis: on above 0.6, off below 0.4. The correct result is 1 switch-on and 0 false. A delay is the time after the true start (20 s) or end (35 s).

- ▶ Hysteresis alone cuts the switch-on events inside the event from 7.51 to 2.39. It does not stop a single false window: such a window is above both thresholds.
- ▶ A filter in time (the mean of 3, or 3 of 5) stops single false windows. “3 of 5” had no false switch-on in 100 runs, for a delay of about 0.2 s.

Key point: Filter the scores in time, and use two thresholds. A delay of some windows is the price.

Source: Simulation of this course (synthetic scores).



- ▶ A **state** is what the system does now. An **event** is an input: a class, a timer, a lost sensor. A **transition** goes from one state to another for one event.
- ▶ Each state has a fixed action: the light is on in ON and off in OFF and in FAULT.
- ▶ Write each state and each event in a table. Each pair needs an answer, also “stay”. A missing pair is a bug.

```
import time
state, last_person, last_data = "OFF", 0.0, time.monotonic()
def step(person, now):
    # called for each new window
    global state, last_person, last_data
    if person is None:
        # no input in this window
        if now - last_data > 10:
            state = "FAULT"
        return state
    last_data = now
    if person:
        last_person = now
    if state == "FAULT":
        state = "OFF"
    if state == "OFF" and person:
        state = "ON"
    elif state == "ON" and now - last_person > 30:
        state = "OFF"
    return state
```

- ▶ The function never waits: no sleep. It compares times, so it can run next to the model and the network code.
- ▶ Use a monotonic clock (`time.monotonic()`, `millis()`). The clock of the day can jump.

Failure	How to detect it	Response
Sensor stops	No reading for a fixed time	Safe state, a fault message
Sensor stuck	The same value for too long, or a value out of range	Ignore the sensor, a fault message
Model unsure	Low scores for many windows	No action, or ask a person
Network lost	No answer, a lost connection	Continue the local decisions, keep the messages
Device crash	No heartbeat; the watchdog timer expires	Restart in the safe state
Power lost	The device restarts	Start in the safe state, read the saved settings

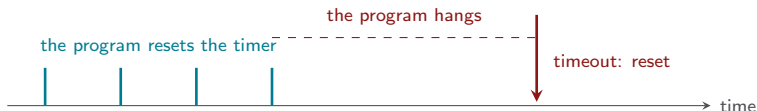
- ▶ Each input needs a timeout. A missing value is an event, not a zero.
- ▶ Plan the response before the failure, and test it: pull a cable, stop the broker, switch the power off.

The safe state depends on the system

System	Safe state	Reason
Heater	Off	A heater that is on with no control can start a fire
Conveyor belt	Stopped	A moving belt with no control can hurt a person
Door of a fire exit	Unlocked	People must leave the building
Door of a store room	Locked	Goods must stay protected
Ventilation of a lab	On	Gas must leave the room

- ▶ “Off” is not always safe. The safe state comes from the risk of the system, not from the code.
- ▶ A safety function that must work in all cases needs a hardware path too: a fuse, a thermal switch, an emergency stop. A model and a program are not enough.

Key point: Write the safe state of each output before you write the rules.



- ▶ A **watchdog timer** is a counter in the chip. The program resets it in each loop. If the program hangs, the counter expires and resets the chip.
- ▶ The ESP32-S3 has three watchdog timers: one in each of the two timer groups and one in the RTC module. Each one can reset the processor or the system.
- ▶ A **heartbeat** is the same idea between devices: a short message each few seconds. No heartbeat for a fixed time means that the device is gone.
- ▶ MQTT has a built-in form of this: the keep-alive and the last will message (Part 2).

Source: ESP32-S3 data sheet, version 2.2, section "Watchdog Timers".

- ▶ **Replay:** record the scores of a real run, then feed the same file to the decision layer. Each change of a rule then has a fair comparison, as in the simulation of this part.
- ▶ **One test for each transition:** a table of the state, the event, and the next state. Each row is one small test.
- ▶ **Fault injection:** stop the sensor, stop the broker, send a value out of range, restart the device. Check the safe state.
- ▶ **Count the switches:** in a test of one hour, how often did the output change? Many changes show a missing filter or a missing hysteresis.

State	Event	Next state	Test
OFF	person	ON	one window with a person
ON	30 s, no person	OFF	a run of 31 s with no person
ON	no data, 10 s	FAULT	stop the input for 11 s
FAULT	data again	OFF	start the input again

A door monitor has two sensors: a contact **D** (door open or closed), and a person detector **P** (a camera model: a person near the door, or not). It must do this:

1. Door closed: nothing to do.
2. Door open, and a person near it: normal.
3. Door open with no person for 30 s: send one alert.
4. The door closes: back to normal, and the alert ends.
5. A sensor sends no value for 10 s: send a fault message.

Write the states, the events, and the transitions as a table. Give the action of each state. Which input needs a filter in time, and which one?

Work with your neighbour for 8 minutes.

Answer: a state machine for a door

State, event	Next state	Action
CLOSED, D open and P	OPEN_PERSON	none
CLOSED, D open, no P	OPEN_ALONE	start the 30 s timer
OPEN_PERSON, no P	OPEN_ALONE	start the 30 s timer
OPEN_ALONE, P	OPEN_PERSON	stop the timer
OPEN_ALONE, timer ends	ALERT	send one alert
any open state or ALERT, D closed	CLOSED	stop the timer; end the alert
any state, no value for 10 s	FAULT	send a fault message
FAULT, both sensors send again	from D and P	the state of the inputs

- ▶ P comes from a model: filter it in time ("3 of 5" windows), or one false window starts or stops the timer.
- ▶ D is a contact: a short debounce of some ms is enough.
- ▶ ALERT stays when a person comes: the door was open and alone for 30 s. Only the closed door ends it.

Part 1 Local decision-making

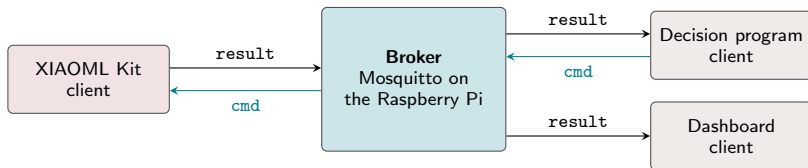
Exercise: write the state machine for a door monitor with two sensors

Part 2 MQTT

Exercise: design the topic tree for ten devices in two rooms

Part 3 Edge-to-cloud design

Exercise: calculate the data of video and of detections for one day



- ▶ Each program is a **client**. A client connects only to the **broker**. It does not know the address of the other clients.
- ▶ A client **publishes** a message to a **topic**. The broker sends the message to each client with a matching **subscription**.
- ▶ A new program needs no change of the others: a second dashboard subscribes, and the XIAO code stays the same.
- ▶ HTTP (Day 11): one client asks one server. MQTT: many senders and many receivers, and one broker between them.

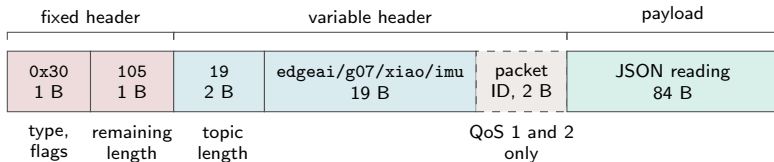
Measured: one session of the lab board

Packet	From	Bytes	Content
CONNECT	client	54	client ID xiao-07, keep-alive 15 s, last will offline
CONNACK	broker	4	connection accepted
PUBLISH	client	32	online to .../xiao/status, retained
SUBSCRIBE	client	26	edgeai/g07/xiao/cmd, QoS 1
SUBACK	broker	5	QoS 1 granted
PUBLISH	client	107	one IMU reading, QoS 0
PUBLISH, PUBACK	both	109 + 4	the same reading, QoS 1
PUBLISH + 3	both	109 + 12	the same reading, QoS 2: PUBREC, PUBREL, PUBCOMP
DISCONNECT	client	2	normal end: no last will

- ▶ MQTT uses one TCP connection for each client: the bytes arrive in order, with no loss.
- ▶ To connect and subscribe costs 4 packets and 89 bytes, one time. Then one IMU reading at QoS 0 is one packet of 107 bytes.
- ▶ Each packet also needs the IP and TCP headers: 40 bytes (Day 11).

Source: Experiment of this course: Mosquitto 2.0.11, a client with the settings of the lab board.

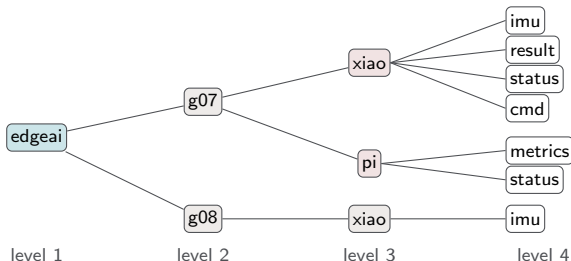
The PUBLISH packet



- ▶ The first byte holds the packet type (3 = PUBLISH) and three flags: DUP (a second copy), QoS (2 bits), and RETAIN.
- ▶ The remaining length uses 7 bits of each byte: 1 byte up to 127, at most 4 bytes. The largest packet has 256 MB.
- ▶ The broker does not read the payload. MQTT does not define its format.
- ▶ The topic is 19 of the 107 bytes (18 percent). A short topic saves bytes in each message.

Source: OASIS, *MQTT Version 3.1.1*, sections 2.2 and 3.3. Sizes from the experiment of this course.

Topics: a tree of levels



- ▶ A topic is a text with levels, separated by /: `edgeai/g07/xiao/imu`. A new topic needs no setup.
- ▶ Topics are case sensitive: `Edgeai` and `edgeai` are two topics. A topic has 1 to 65 535 bytes of UTF-8.
- ▶ A leading / makes an empty first level: `/edgeai/g07` is a different topic. Do not use it.
- ▶ The broker owns the topics that start with \$: `$SYS/...` gives its statistics.

Source: OASIS, *MQTT Version 3.1.1*, section 4.7.

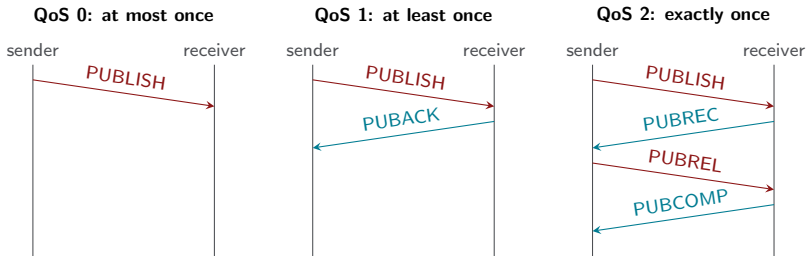
Filter	Matches	Does not match
edgeai/g07/xiao/imu	only this topic	edgeai/g07/xiao/result
edgeai/g07/+/status	.../xiao/status, .../pi/status	edgeai/g07/xiao/imu
edgeai/+/xiao/imu	the IMU topic of each group	edgeai/g07/pi/metrics
edgeai/g07/#	each topic of group 7, also edgeai/g07	edgeai/g08/xiao/imu
#	each topic	\$SYS/...

- ▶ + matches exactly one level. # matches zero or more levels, and it must be the last character.
- ▶ Not valid: edgeai/#/imu and edgeai/g0+. A wildcard fills a complete level. The character * is not a wildcard in MQTT.
- ▶ Only a subscription has wildcards. A published topic has none.
- ▶ A wide filter costs: the board receives and parses each message that matches. Subscribe a board only to its own topics.

Source: OASIS, *MQTT Version 3.1.1*, section 4.7.1.

1. Fix the order of the levels, from general to specific. The lab: `edgeai/<group>/<device>/<channel>`.
2. Put the identity in the topic and the data in the payload. A topic has no time stamp and no sequence number.
3. Use one topic for each type of message. A program then selects its messages with one filter.
4. Separate the two directions: data up (`imu`, `result`, `status`), commands down (`cmd`).
5. Start from the subscriptions. Write the filter that each program needs. Many filters for one program show a bad order of the levels.

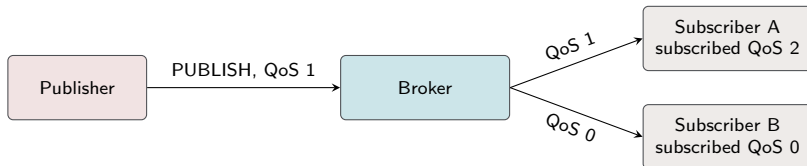
Program	Filter
XIAOML Kit of group 7	<code>edgeai/g07/xiao/cmd</code>
Decision program on the Pi	<code>edgeai/g07/xiao/result</code>
Dashboard of group 7	<code>edgeai/g07/#</code>
Instructor laptop	<code>edgeai/+/#/status</code>



- ▶ **QoS 0**: one packet, no answer. A lost connection loses the message.
- ▶ **QoS 1**: the sender keeps the message until the PUBACK, and sends it again with the DUP flag. The receiver can get two copies.
- ▶ **QoS 2**: the receiver keeps the packet ID until the PUBREL, so it can see a second copy and drop it. Four packets for each message.
- ▶ The two sides of a hop agree on the QoS of each message. The sender selects it in each PUBLISH.

Source: OASIS, *MQTT Version 3.1.1*, section 4.3.

The QoS applies to one hop



- ▶ The publisher sets the QoS of the first hop. The subscription sets the largest QoS of the second hop. The broker sends with the lower of the two values.
- ▶ A PUBACK says: the broker has the message. It does not say that a subscriber has it, or that a program acted on it.
- ▶ For a confirmation from the application, the receiver publishes a reply: a command on `.../xiao/cmd` gets an answer on `.../xiao/ack` with the same command ID.

Source: OASIS, *MQTT Version 3.1.1*, section 3.8.4.

Measured: the cost of each QoS level

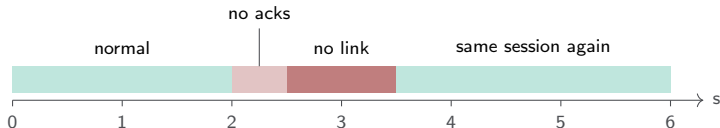
QoS	Packets	Bytes	Median time		Messages each second	
			to the ack	to the subscriber	1 in flight	20 in flight
0	1	147	0.4 ms	10.9 ms	above 7000	
1	2	193	21.5 ms	10.9 ms	47	551
2	4	281	42.4 ms	32.4 ms	24	241

Bytes: one IMU reading of the lab with the IP and TCP headers. The proxy added 10 ms to each direction of the publisher link: a round trip of 20 ms.

- ▶ QoS 1 waits one round trip for its PUBACK, and QoS 2 waits two round trips.
- ▶ Mosquitto gives a QoS 2 message to the subscribers only after the PUBREL: after three trips over the link.
- ▶ With one message in flight, QoS 1 sends one message in each round trip: 47 each second. A window of 20 messages gave 551.
- ▶ Use QoS 0 for a fast stream, and QoS 1 for a message that must arrive.

Source: Experiment of this course (one computer, 200 messages for each time, 1000 for each rate).

Measured: a lost link



QoS	Lost	Double	What happened
0	51	0	The client dropped each message while it had no connection.
1	0	20	The client sent the 20 messages with no PUBACK again, with the DUP flag.
2	56	0	See the next frame.
2, broker limit 100	0	0	Exactly once, as the standard says.

- ▶ 50 messages each second for 6 s: 300 messages. A subscriber with QoS 2 counted the sequence numbers.
- ▶ QoS 1 gives no loss but doubles: the receiver drops a second copy by its seq.

Source: Experiment of this course (Mosquitto 2.0.11, paho-mqtt 2.1.0).

1. During the cut, the client kept 20 messages in flight and more than 50 new messages in its memory.
 2. After the new connection, paho-mqtt 2.1.0 sent all of them at once. Its count of messages in flight starts again at 0 after a reconnection.
 3. Mosquitto accepted at most 20 QoS 2 messages in flight from this client (`max_inflight_messages`, default 20). It refused the next 56 with the reason code 0x97, "quota exceeded".
 4. A PUBREC of MQTT 3.1.1 has no field for a reason code. The client saw a normal PUBREC and ended the flow. No program reported an error.
- With the limit at 100, the same test lost no message. MQTT 5 has the reason code in the PUBREC, so a client can see the refusal.

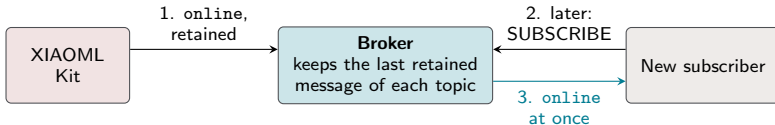
Key point: A QoS level is a promise of the protocol, and each library has limits. Number the messages, count them at the receiver, and test with a lost link.

Source: Experiment of this course; OASIS, *MQTT Version 5.0*, section 2.4.

Subscriber that was away	QoS 0 messages	QoS 1 messages
Persistent session, subscription QoS 1	0 of 10	10 of 10
Clean session, subscription QoS 1	0 of 10	0 of 10
Persistent session, subscription QoS 0	0 of 10	0 of 10

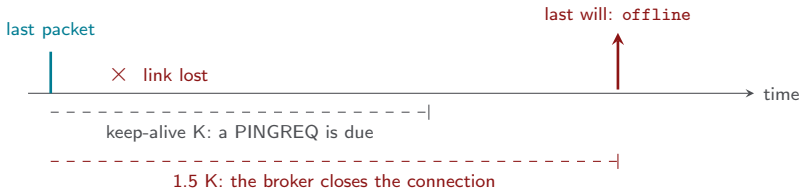
- ▶ The CONNECT packet has the flag “clean session”. With the flag off, the broker keeps the subscriptions of the client and its QoS 1 and QoS 2 messages while it is away. The CONNACK says “session present”.
- ▶ The session belongs to the client ID. Each device needs a fixed and unique ID: a second client with the same ID disconnects the first one.
- ▶ The broker does not keep QoS 0 messages (Mosquitto option `queue_qos0_messages`, default off). A subscription with QoS 0 makes each message QoS 0.
- ▶ The store has a limit: Mosquitto keeps at most 1000 messages for each client. MQTT is not a database.

Source: Experiment of this course. OASIS, *MQTT Version 3.1.1*, sections 3.1.2.4 and 3.1.4; Mosquitto 2.0.11 manual.



- ▶ Measured: a new subscriber to `edgeai/g07/#` got the retained status `online` with the `retain` flag at once. It did not get the IMU message that was sent before and not retained.
- ▶ Use a retained message for a state or a setting: the status of a device, a threshold, a mode.
- ▶ Do not retain a command or an event. Measured: a retained `led=1` came again at each of 3 new connections.
- ▶ To delete a retained message, publish an empty retained message to its topic. Measured: the next subscriber got nothing.

Source: Experiment of this course. OASIS, *MQTT Version 3.1.1*, section 3.3.1.3.



- ▶ **Keep-alive** K: a client that sends nothing for K seconds sends a PINGREQ (2 bytes), and the broker answers with a PINGRESP.
- ▶ With no packet for 1.5 K, the broker closes the connection.
- ▶ **Last will**: a topic and a message in the CONNECT packet. The broker publishes it when the connection ends with no DISCONNECT packet: a lost link, a crash, no power.
- ▶ `umqtt.simple` has $K = 0$ (off) by default: the broker then never finds a lost board. The lab code sets 15 s.

Source: OASIS, *MQTT Version 3.1.1*, sections 3.1.2.5 and 3.1.2.10; code of `umqtt.simple` (micropython-lib).

End of the connection	Last will after	Rule: 1.5 K
Link lost, keep-alive 5 s	8.4 to 11.1 s	7.5 s
Link lost, keep-alive 10 s	16.9 to 17.6 s	15 s
Link lost, keep-alive 15 s	22.8 to 23.4 s	22.5 s
Process ends, no DISCONNECT	0.6 ms	
Normal DISCONNECT	no last will	

Three runs for each keep-alive. “Link lost”: the proxy stopped all traffic and kept the TCP connection open.

- ▶ A crashed process closes its TCP connection: the broker knows at once. A lost Wi-Fi link sends nothing: only the keep-alive finds it. Mosquitto needed up to 3.6 s more than 1.5 K.
- ▶ A short K finds a lost device faster, but costs bytes and energy: with $K = 15$ s and no other traffic, 20 160 bytes each hour.
- ▶ The status pattern: the last will *offline* is retained, and the client publishes *online* retained after each connection. Each program knows the state of each device.

Source: Experiment of this course. Bytes: PINGREQ and PINGRESP with the IP and TCP headers.

```
import json
import paho.mqtt.client as mqtt

base = "edgeai/g07"
c = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="pi-07",
               clean_session=False)
c.will_set(base + "/pi/status", "offline", qos=1, retain=True)
def on_connect(client, userdata, flags, reason, props):
    client.publish(base + "/pi/status", "online", qos=1, retain=True)
    client.subscribe(base + "/xiao/result", qos=1)
def on_message(client, userdata, msg):
    r = json.loads(msg.payload)
    if r["label"] == "yes" and r["score"] > 0.8:
        client.publish(base + "/xiao/cmd", "led=1", qos=1)
c.on_connect, c.on_message = on_connect, on_message
c.connect("localhost", 1883, keepalive=15)
c.loop_forever()                                # reconnects after a lost link
```

- ▶ `on_connect` runs after each new connection: the status and the subscription come back after a lost link.
- ▶ Keep `on_message` short. The network loop waits while it runs, so a slow callback delays the keep-alive.

Message	Way	QoS	Retained	Example
Telemetry stream	up	0	no	IMU reading, 5 each second
Result, event	up	1	no	"label":"yes"
Status	up	1	yes, last will	online, offline
Setting	down	1	yes	a threshold
Command	down	1	no	led=1
Reply	up	1	no	"id":17,"ok":true

Up: from the device to the server. Down: from the server to the device.

- ▶ A stream can lose one reading: the next one comes soon. An event or a command must arrive.
- ▶ Give each command an ID, and write it as a state: led=1, not "toggle". A second copy then does no harm.
- ▶ A command for a device that is away: drop it, or keep only the newest. Commands in a fixed order need sequence numbers.

Format of one IMU reading	Payload	PUBLISH	With IP, TCP
JSON (format of the lab)	84	107	147
CSV text	41	64	104
CBOR, float32 values	60	83	123
Binary, float32 values	32	55	95
Binary, int16 values	20	43	83
JSON, 10 readings in one message	851	875	915

Bytes. Topic of 19 bytes, QoS 0.

- ▶ MQTT carries bytes. The sender and the receiver must agree on the format.
- ▶ JSON: each tool and each person can read it. It repeats the field names in each message.
- ▶ Binary int16 is 4.2 times smaller as a payload, but only 1.8 times smaller on the link: the headers stay.
- ▶ 10 readings in one message cut the bytes for each reading from 147 to 91.5. The cost is delay.

Source: Calculation of this course (seq, ms, 6 IMU values).

```
{"seq":12,"ms":34567,"label":"yes","score":0.91}
```

- ▶ A **sequence number** seq: the receiver finds lost, double, and old messages. It starts again at 0 after a restart.
- ▶ A **time**: ms is the time since the start of the board. A clock time needs a synchronized clock (Part 3).
- ▶ **Units** in the field name or in a written schema: temp_c, not temp.
- ▶ A **version** field when the format can change: a new firmware can then add a field, and old receivers still work.
- ▶ **Check** each message on receipt: the type and the range of each field. Count and drop a bad message, as the guard of Day 10.
- ▶ A **size limit**: PubSubClient holds a complete packet in a buffer of 256 bytes.

Source: Payload of the lab board (hardware folder HW-07); PubSubClient 2.8, PubSubClient.h.

Software	Role	QoS	Limits
Mosquitto 2	broker on the Raspberry Pi	0, 1, 2	only local clients with no listener line; 20 messages in flight, 1000 kept for each client
paho-mqtt 2.1	Python client	0, 1, 2	MQTT 3.1.1 and 5; reconnects in <code>loop_forever()</code>
PubSubClient 2.8	Arduino client	publish 0; subscribe 0, 1	buffer of 256 bytes; keep-alive 15 s
umqtt.simple	MicroPython client	0, 1	keep-alive off by default; no reconnection

- ▶ The lab uses MQTT 3.1.1 (OASIS, 2014). MQTT 5 (OASIS, 2019) adds reason codes, a time limit for each message, and shared subscriptions.
- ▶ Ports: 1883 with no encryption, 8883 with TLS.
- ▶ The lab network has no password. Module SY-2 adds user names, passwords, and TLS.

Source: Mosquitto 2.0.11 manual; code of PubSubClient 2.8 and umqtt.simple; OASIS, *MQTT Version 3.1.1*, section 4.2.

Exercise: a topic tree for two rooms

A building has two rooms, **A** and **B**. Each room has five devices: two XIAOMI Kits (IMU and keyword model), one Raspberry Pi with a camera (it counts people), one temperature sensor, and one fan.

1. Write the topic tree for the ten devices: data, results, status, and commands.
2. Write the filter for: (a) a dashboard of all temperatures, (b) the fan controller of room A, (c) a monitor of the status of all devices.
3. Give the QoS and the retained flag for each type of topic.

Work with your neighbour for 8 minutes.

Answer: a topic tree for two rooms

Tree: site/<room>/<device>/<channel>, for example site/a/temp-1/temp_c, site/a/xiao-2/result, site/b/fan-1/cmd.

Program	Subscribes to	Publishes to
(a) Temperature dashboard	site/+/+/temp_c	none
(b) Fan controller, room A	site/a/+/temp_c, site/a/pi-1/count	site/a/fan-1/cmd
(c) Status monitor	site/+/+/status	none

Topic type	QoS	Retained
imu, temp_c	0	no
result, count	1	no
status	1	yes, also the last will
cmd	1	no

- ▶ The channel name says the type: one filter selects all readings of one type. The order room, then device, suits a controller for each room.

Part 1 Local decision-making

Exercise: write the state machine for a door monitor with two sensors

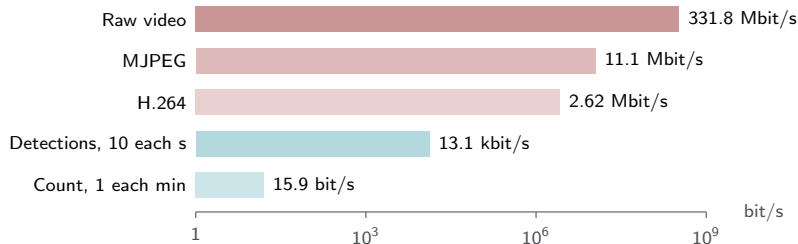
Part 2 MQTT

Exercise: design the topic tree for ten devices in two rooms

Part 3 Edge-to-cloud design

Exercise: calculate the data of video and of detections for one day

Send results, not raw data



- ▶ One camera, 1280×720 at 30 frames each second. Each step on the device makes the data smaller. The scale is logarithmic.
- ▶ The detection messages need 200 times fewer bits than the H.264 stream. No image leaves the device: this also protects privacy.
- ▶ The cost: the server cannot look at the video again. Keep a short ring buffer of images on the device, and send a sample when a person must check a result.

Source: MJPEG and H.264: Day 11 measurements. Messages: calculation of this course (164 bytes for each detection message, 119 bytes for each count, with IP and TCP).

One day of the keyword model

What the XIAO sends	Messages	Bytes in one day	Raw / this
Raw audio, 16 kHz, 16 bits		2 764 800 000	1
One result each 0.5 s	172 800	20 908 800	132
Only the keywords (50 in one day)	50	6 050	457 000

One result message: 55 bytes of JSON, 121 bytes with the topic and the MQTT, IP, and TCP headers.

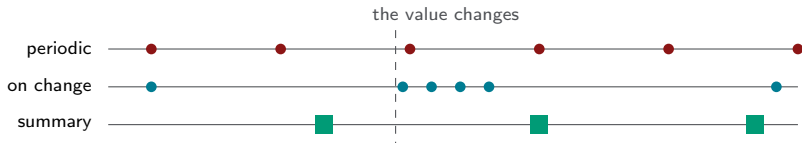
- ▶ 2.76 GB of audio in one day does not fit the Wi-Fi budget of a small device, and the server does not need it.
- ▶ One result for each window (Day 5: one window each 0.5 s) is useful for a test: you can see each score.
- ▶ The application needs only the events. Fewer messages also keep the radio off for longer: this saves energy (Day 9).
- ▶ Send more detail for a short time when you look for an error, then go back to the events.

Source: Calculation of this course. 50 keywords in one day is an example value.

Type	Example	Use
Result	a class and a score for each window	tests, a live view
Event	“a person came”, “door open”	actions and alerts
Summary	count, mean, minimum, maximum each minute	trends, reports, dashboards (Day 13)
Status	online, versions, free memory, temperature	monitoring of the device
Sample	a few images when the model is not sure	a check by a person, new training data

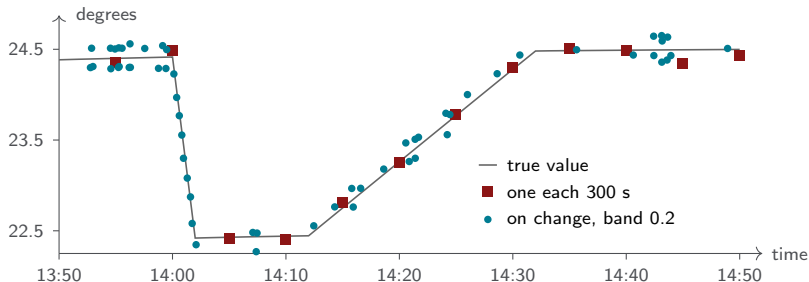
- ▶ Start from the questions of the receiver. Each message type answers one question.
- ▶ Raw data stays on the device in a ring buffer: the newest minutes only. A request from the server can get one part of it.
- ▶ A sample of the uncertain inputs is the start of the loop back to training (Day 13 and Day 14).

How often to send telemetry



- ▶ The answer depends on the task. A thermostat needs one reading each few minutes. A machine that can break needs several readings each second.
- ▶ Three questions: how fast does the value change? How fast must the receiver act? What does one message cost in bytes and energy?
- ▶ Sample fast, send slow. The sampling rate (Day 2) is not the reporting rate. The device can read each second and send each minute.
- ▶ **On change:** send when the value moves more than a band, and a heartbeat after a fixed time with no message. The receiver then knows that the device is alive.

Simulated: one hour around an event



- ▶ At 14:00 a door opens: the temperature falls by 2 degrees in 2 minutes and comes back in 20 minutes.
- ▶ The rule “one each 300 s” shows the fall only at 14:05. The rule “on change” sends a burst during the fall, and few messages when the value is stable.

Source: Simulation of this course (synthetic signal, noise of 0.05 degrees, seed 0).

Simulated: five rules for one day

Rule	Messages	Bytes	Largest error	Event seen
Each reading (1 s)	86 400	9 222 577	0.24	58 s
One each 60 s	1 440	151 347	1.03	119 s
One each 300 s	288	30 094	2.06	299 s
On change (0.2), heartbeat 300 s	964	101 013	0.24	58 s
Summary each 60 s	1 440	183 027	1.47	59 s

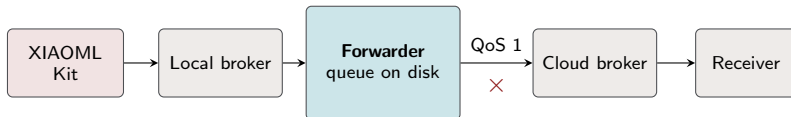
Error: the last sent value against the true value, in degrees. Event seen: the first message 1 degree below the value before the event. Summary: the error uses the mean, the event uses the minimum.

- ▶ “On change” sent 90 times fewer messages than “each reading”, with the same largest error and the same delay.
- ▶ A long period saves bytes but hides a short event: 299 s of delay and an error of 2.06 degrees.
- ▶ A summary with the minimum and the maximum shows the event after 59 s with one message each minute.

Source: Simulation of this course (synthetic signal of one day, seed 0). Bytes with the topic and the MQTT, IP, and TCP headers.

Choice	When	Example
Drop	only the newest value matters	the temperature for a thermostat
Keep the newest	a state or a setting	a retained status, a threshold
Store and forward	each message matters	counts, events, the data of a trend

- ▶ A heating system does not need the temperature of 20 minutes ago. A model that looks for a change in a trend needs each value.
- ▶ Commands to a device that is away: drop the old ones, or keep only the newest. Commands that must run in order need a queue with sequence numbers.
- ▶ The local decisions continue in all three cases (Part 1). Only the messages to the cloud wait.



1. write 2. send the oldest 3. delete after the PUBACK

- ▶ Write each message to the queue before you send it. Delete it only after the PUBACK of the cloud broker.
- ▶ A message with no PUBACK is sent again: the receiver can get two copies. It keeps the pair (device, seq) and drops a second copy.
- ▶ After a long cut, the receiver gets old messages. It uses the time stamp of the device, not the time of arrival.
- ▶ The queue must be on disk, not only in memory: the device can restart during the cut.

Measured: a cloud outage of 60 s

Way to the cloud	Lost	Double	First after	Oldest age
Python forwarder, SQLite	0	0	0.12 s	60.1 s
Mosquitto bridge, defaults	0	0	20.5 s	80.5 s
Bridge, a queue of 200	547	0	14.7 s	74.7 s

10 messages each second for 120 s (1200 messages, QoS 1); no link from 20 s to 80 s (600 messages). “First after”: the first message at the receiver after the link came back.

- ▶ The forwarder sent the backlog of 600 messages in 0.44 s after the link came back.
- ▶ The bridge tries to connect again after a random wait of 5 to 30 s (default `restart_timeout`): the backlog waited 20.5 s.
- ▶ The bridge keeps messages in memory up to `max_queued_messages`. With 200, it dropped 547 messages: 400 of the cut and 147 of the wait after it.

Source: Experiment of this course: two Mosquitto 2.0.11 brokers and paho-mqtt 2.1.0 on one computer.

1. Each commit waits for the disk.

SQLite, one INSERT and one commit	Median	p95
Default settings	2.8 ms	3.9 ms
journal_mode=WAL, synchronous=NORMAL	0.027 ms	0.035 ms

- ▶ With the defaults, the queue writes at most about 350 messages each second on this disk. A microSD card is slower.

2. A new row can get the ID of a deleted row.

- ▶ The first forwarder of this experiment marked the sent rows by their ID. SQLite gave new messages the IDs of deleted ones: 1160 of 1200 messages were never sent.
- ▶ Use AUTOINCREMENT, or a column “sent” in the row.

Key point: Test the buffer with a real cut of the link, and count each message at the receiver.

Source: Experiment of this course (200 commits for each setting).

A Mosquitto bridge on the Raspberry Pi

```
# /etc/mosquitto/conf.d/bridge.conf on pi-07
connection cloud
address 192.168.8.10:1883
topic edgeai/g07/# out 1
cleansession false
remote_clientid pi-07-bridge
max_queued_messages 100000
persistence true
```

- ▶ The bridge is a client of the cloud broker. It sends each local message of edgeai/g07/# with QoS 1 (out: only from the Pi to the cloud).
- ▶ With cleansession false, the cloud broker keeps the session of the bridge during a cut.
- ▶ max_queued_messages is 1000 by default. Set it from the longest cut that you plan for.
- ▶ With persistence true, Mosquitto saves its queue to a file when it stops and each 30 minutes. A power cut loses the newest messages.

Source: Mosquitto 2.0.11 manual (mosquitto.conf). Address of the instructor laptop: hardware folder HW-09.

buffer = messages each second $\times$ longest cut $\times$ bytes of one message

Device	Rate and cut	Buffer
XIAO, results	2 each s, 10 minutes	$1200 \times 60 \text{ B} = 72 \text{ kB}$
Raspberry Pi, detections	10 each s, 1 day	$864\,000 \times 100 \text{ B} = 86.4 \text{ MB}$

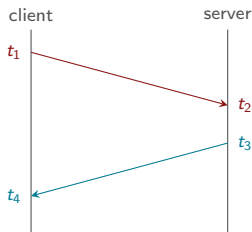
Example values of this course. 1 kB = 1000 bytes.

- ▶ The Raspberry Pi has a microSD card of many GB: one day is easy. The XIAO has 8 MB of PSRAM: some minutes of results, or a summary.
- ▶ When the buffer is full, select a rule: drop the oldest, drop the newest, keep one message of each N, or keep only summaries.
- ▶ Flash memory wears with each write: write the queue in blocks, not one message at a time.

Clock	Example	Property
Time since boot	<code>millis()</code> , <code>time.monotonic()</code>	never jumps; starts at 0 after each restart
Clock time (UTC)	<code>time.time()</code>	needs a synchronization; can jump when it is set
Arrival time	the clock of the server	adds the network delay and the time in a buffer

- ▶ Put the time stamp on the message at the source, when the value is measured. After a cut, the arrival time can be minutes late.
- ▶ Use UTC in milliseconds: no time zones, no summer time.
- ▶ Measure a duration with the time since boot (Part 1). Name an instant with the clock time.
- ▶ Order the messages of one device by seq. A clock that jumps does not change this order.

NTP: set a clock over the network



$$\text{offset} = \frac{(t_2 - t_1) + (t_3 - t_4)}{2}$$

$$\text{delay} = (t_4 - t_1) - (t_3 - t_2)$$

- ▶ The formulas assume the same time in the two directions. The error of the offset is at most half of the delay.
- ▶ Measured: 10 requests from the work computer to `pool.ntp.org`: delay 68 to 98 ms, offset 0.2 to 5.1 ms. Its clock was within some ms of the server.
- ▶ A device asks again at intervals, because its clock drifts. A clock with an error of 20 ppm (parts per million) drifts 1.7 s each day.

Source: RFC 5905 (NTP version 4), section 8. Measured by this course. 20 ppm is an example value.

- ▶ In the lab, the internet link can fail. Then no device reaches `pool.ntp.org`.
- ▶ The Raspberry Pi is the time server of its group: `chrony` with the lines `allow 192.168.8.0/24` and `local stratum 10` serves its clock to the lab network.
- ▶ The Raspberry Pi 5 has a real-time clock. With a battery on the connector J5, it keeps the time with no power.
- ▶ The XIAO asks the Raspberry Pi at the start and each hour: `configTime(0, 0, "192.168.8.107")`.
- ▶ The XIAO has no battery for its clock. After a restart, it has no clock time until the first answer.
- ▶ Until then, the device sends `ms` since the start and a boot counter, and it marks the message "not synchronized". The receiver can then calculate the time from a later message.

Source: Raspberry Pi documentation, "Real Time Clock (RTC)". Addresses from the hardware folder HW-09.

- ▶ Each device has one ID that does not change: the MAC address in the eFuse of the ESP32-S3 (`ESP.getEfuseMac()`), the serial number of the Raspberry Pi.
- ▶ The same ID is in the client ID, in the topic, and in the status. A copied firmware must not copy the ID.
- ▶ Each device or group has its own password. The broker then limits each one to its own topics:

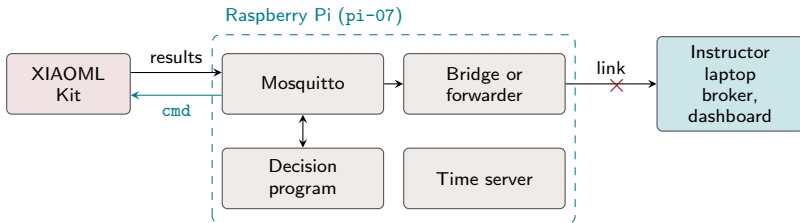
```
# Mosquitto ACL file: user g07 can use only edgeai/g07/...  
pattern readwrite edgeai/%u/#
```

- ▶ The retained status names the device and its versions, so the server knows which model gave each result:

```
{"id":"xiao-34851a2b3c4d","fw":"1.2.0","model":"kws-3","boot":17}
```

Source: Mosquitto 2.0.11 manual (ACL patterns). Module SY-2 adds the passwords and TLS.

The edge-to-cloud design of the lab



- ▶ The XIAO and the Raspberry Pi form the local system: the decision and the command do not need the link to the laptop.
- ▶ The link to the laptop is the “cloud” of the lab. In Part D you cut it, and the local action must continue.
- ▶ After the link comes back, the receiver counts the seq numbers: no message may be lost.

Exercise: the data of one camera for a day

A camera node on a Raspberry Pi runs a person detector at 10 frames each second. Calculate the data that it sends in one day (86 400 s) for each design:

1. The H.264 video stream at 2 Mbit/s.
2. One result message for each processed frame. One message is 150 bytes with all headers.
3. One message only when the number of persons changes: 200 changes in one day, 150 bytes each.

Give each result in bytes, and the ratio of design 1 to designs 2 and 3. Which design would you select for a dashboard of the people in a room?

Work with your neighbour for 8 minutes. Use $1 \text{ GB} = 10^9$ bytes.

Answer: the data of one camera for a day

Design	Calculation	One day
1. H.264 video	$2\,000\,000 / 8 \times 86\,400$	21.6 GB
2. A result each frame	$10 \times 150 \times 86\,400$	129.6 MB
3. Only the changes	200×150	30 kB

- ▶ Design 1 sends 167 times the data of design 2, and 720 000 times the data of design 3.
- ▶ For a dashboard of the people in a room, design 3 is enough: add a heartbeat, so the dashboard knows that the node is alive.
- ▶ Keep design 2 for a short test, and a few images for a check by a person. The video stays on the node.

- ▶ A decision that must happen within a fixed time, or with no network, stays on the device. A rule or a state machine with hysteresis makes it stable, and a safe state handles a failed part.
- ▶ An MQTT broker gives each message to the subscribers of its topic. A topic tree with a fixed order and the wildcards `+` and `#` lets each program select its messages.
- ▶ A higher quality of service (0, 1, 2) sends more messages and adds delay, and it lowers the risk of a lost or a double message. A retained message gives the last value to a new subscriber. The last will tells the others that a device is gone.
- ▶ An edge device sends results, not raw data. It keeps messages in a buffer when the link fails, sends them later with a sequence number and a time stamp, and has its own identity.

Key point: Decide locally what must not wait. Send small results with MQTT, and design for a link that fails.

Book chapters

- ▶ *Machine Learning Systems* (mlsysbook.ai), Volume I, chapter “ML Systems”, section “Hybrid Architectures”.
- ▶ *XIAO: Big Power, Small Board*, chapter 3.5: “Telemetry and Commands using the MQTT protocol”.

Standards

- ▶ OASIS, “MQTT Version 3.1.1”, 2014.
- ▶ OASIS, “MQTT Version 5.0”, 2019.

Tools

- ▶ Mosquitto: <https://mosquitto.org/documentation/>
- ▶ Eclipse Paho Python client: <https://eclipse.dev/paho/>
- ▶ MicroPython umqtt.simple:
<https://github.com/micropython/micropython-lib>

This lecture adapts material from these sources:

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *XIAO: Big Power, Small Board* by Lei Feng and Marcelo Rovai. github.com/Mjrovai/XIAO_Big_Power_Small_Board-ebook, GPL-3.0.

Material from a source with the licence CC BY-NC-SA 4.0 keeps that licence.